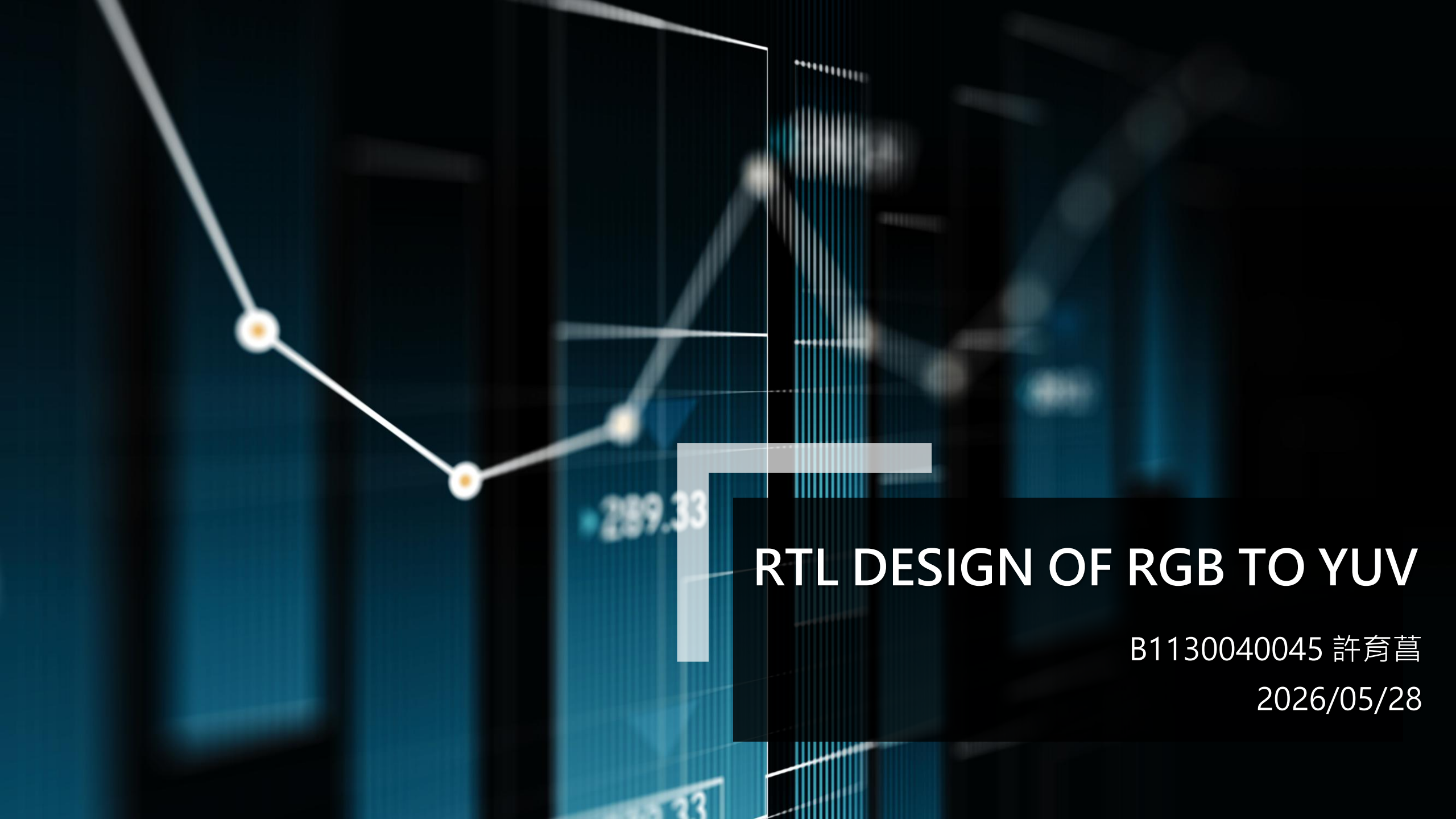




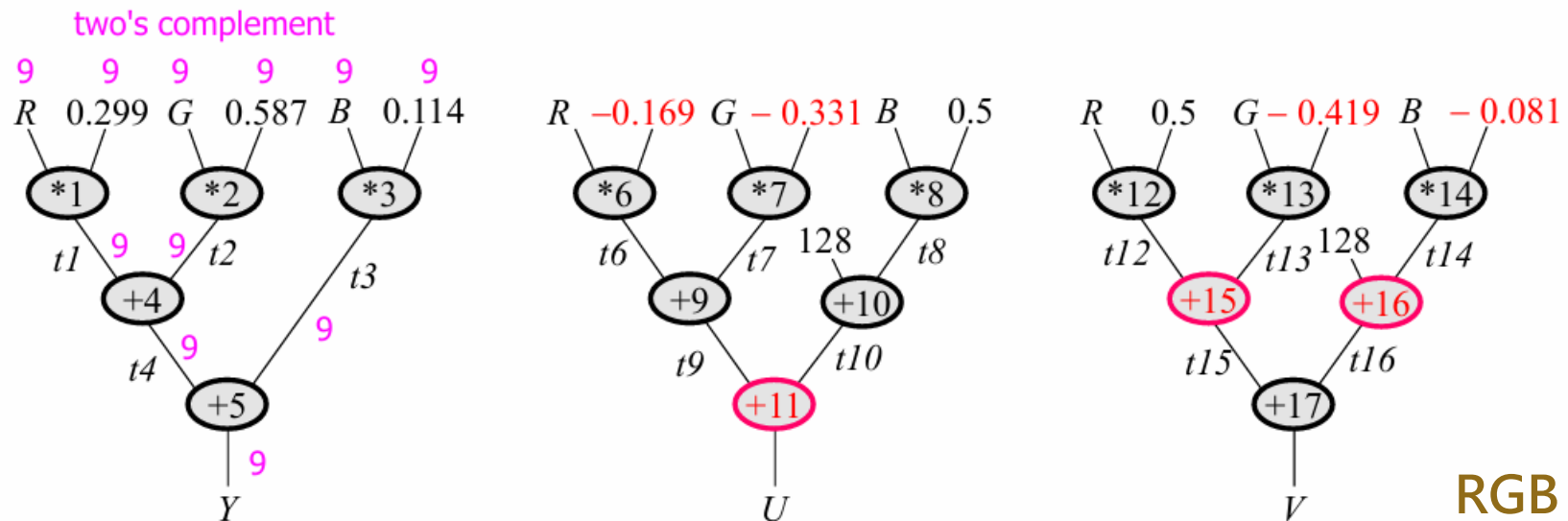
RTL DESIGN OF RGB TO YUV

B1130040045 許育菴

2026/05/28

題目說明

- 使用Synopsys VCS (Verilog Compiler Simulator)以及Verdi，並且根據投影片提供的程式碼，撰寫 RGB to YUV 電路的Verilog code。
- 需使用MobaXterm登入實驗室伺服器來編譯並執行程式碼。



ROM.v

負責儲存RGB to YUV公式
中所出現的**常數**。

使用Fixed-Point表示法，
最左側的bit為sign bit，將
其餘8bit化成十進制，再除
以**256 (2^8)** 即為所表示的
數字。

```
1  module ROM (clk, addr, data);
2      input clk;
3      input [2:0] addr;
4      output reg [8:0] data;
5
6      // 9-bit signed fixed-point constants, scaled by 256.
7      always @(*) begin
8          case(addr)
9              // constants
10             3'b000: data = 9'b001001100; // 0.299 ~= 76 / 256
11             3'b001: data = 9'b010010110; // 0.587 ~= 150 / 256
12             3'b010: data = 9'b111010101; // -0.169 ~= -43 / 256
13             3'b011: data = 9'b110101100; // -0.331 ~= -84 / 256
14             3'b100: data = 9'b010000000; // 0.5 = 128 / 256
15             3'b101: data = 9'b110010101; // -0.419 ~= -107 / 256
16             3'b110: data = 9'b111101011; // -0.081 ~= -21 / 256
17             3'b111: data = 9'b000011101; // 0.114 ~= 29 / 256
18             default: data = 9'b000000000;
19         endcase
20     end
21 endmodule
22
```

datapath.v

建立register，用於暫存state之間的值。

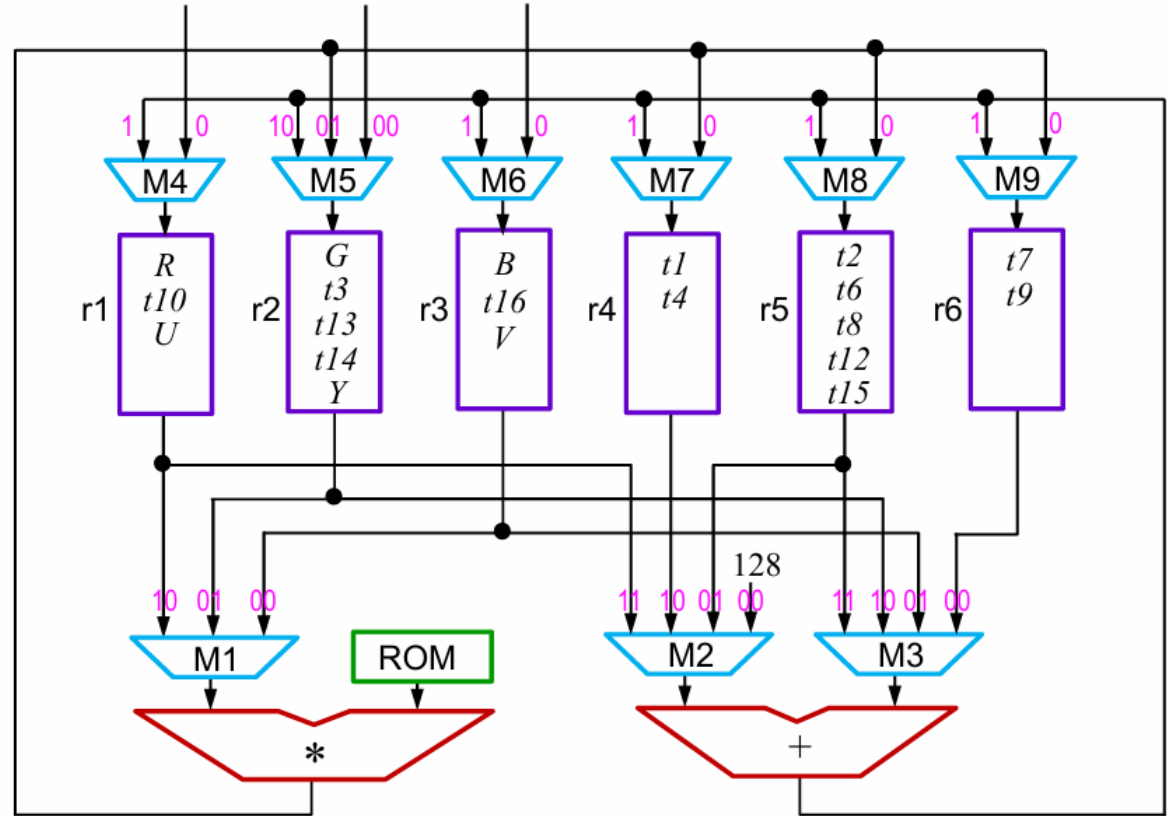
當control[21]的值為1時，將M4_OUT儲存進R1。

```
// control = {r1,r2,r3,r4,r5,r6,    ROM[2:0],    M1[1:0],    M2[1:0],    M3[1:0],M4,M5[1:0],M6,M7,M8,M9}  
//           21 20 19 18 17 16      15 14 13      12 11      `10 9      `8 7      6 5 4      3 2 1 0
```

```
23     Register r1(.D(M4_OUT), .reset(rst_n), .clk(clk), .load(control[21]), .Q(R1));  
24     /*  
25     equal to:  
26         if(reset)  
27             R1 <= 0  
28         else if(control[21])  
29             R1 <= M4_OUT  
30     */  
31     Register r2(.D(M5_OUT), .reset(rst_n), .clk(clk), .load(control[20]), .Q(R2));  
32     Register r3(.D(M6_OUT), .reset(rst_n), .clk(clk), .load(control[19]), .Q(R3));  
33     Register r4(.D(M7_OUT), .reset(rst_n), .clk(clk), .load(control[18]), .Q(R4));  
34     Register r5(.D(M8_OUT), .reset(rst_n), .clk(clk), .load(control[17]), .Q(R5));  
35  
36     Register r6(.D(M9_OUT), .reset(rst_n), .clk(clk), .load(control[16]), .Q(R6));
```

datapath.v

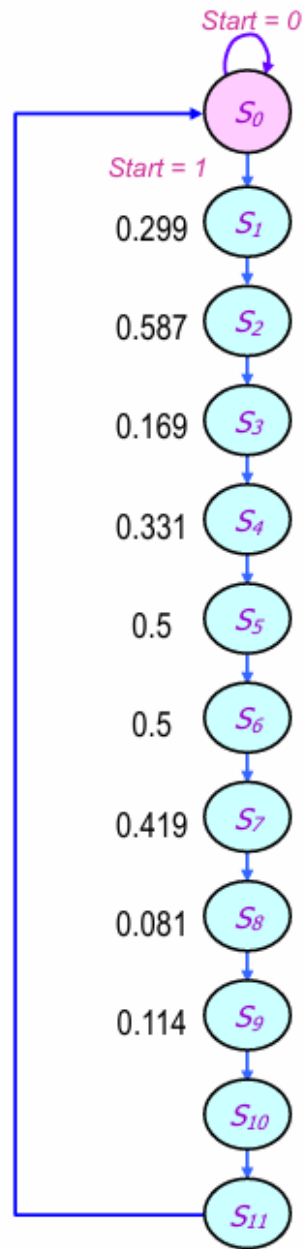
根據右上的電路圖以及對應的control signal，完成MUX M4 ~ M9。



```
// Functional unit input selection.
MUX3 M1(.A(R3), .B(R2), .C(R1), .S(control[12:11]), .Y(M1_OUT));
MUX4 M2(.A(9'b01000000), .B(R5), .C(R4), .D(R1), .S(control[10:9]), .Y(M2_OUT));
MUX4 M3(.A(R6), .B(R3), .C(R2), .D(R5), .S(control[8:7]), .Y(M3_OUT));

// Register input selection.
MUX2 M4(.A(inportR), .B(Fadd), .S(control[6]), .Y(M4_OUT));
MUX3 M5(.A(inportG), .B(Fmul), .C(Fadd), .S(control[5:4]), .Y(M5_OUT));
MUX2 M6(.A(inportB), .B(Fadd), .S(control[3]), .Y(M6_OUT));
MUX2 M7(.A(Fmul), .B(Fadd), .S(control[2]), .Y(M7_OUT));
MUX2 M8(.A(Fmul), .B(Fadd), .S(control[1]), .Y(M8_OUT));
MUX2 M9(.A(Fmul), .B(Fadd), .S(control[0]), .Y(M9_OUT));
```

Controller.v



Present State	Input	Next State	Control Signals							
			r1	r2	r3	r4	r5	r6	ROM	done
S0	Start = 0	S0	1	1	1	0	0	0	-	0
	Start = 1	S1								
S1	-	S2	0	0	0	1	0	0	000	0
S2	-	S3	0	0	0	0	1	0	001	0
S3	-	S4	0	0	0	1	1	0	010	0
S4	-	S5	0	0	0	0	0	1	011	0
S5	-	S6	0	0	0	0	1	1	100	0
S6	-	S7	1	0	0	0	1	0	100	0
S7	-	S8	1	1	0	0	0	0	101	0
S8	-	S9	0	1	0	0	1	0	110	0
S9	-	S10	0	1	1	0	0	0	111	0
S10	-	S11	0	1	0	0	0	0	-	0
S11	-	S0	0	0	1	0	0	0	-	1

state -> control signal 對應表

Controller.v

根據上一頁的表格，依序填入每個state所需要的
control signal。

```
52 case(Current_State)
53     // control = {r1,r2,r3,r4,r5,r6,ROM,M1,M2,M3,M4,M5,M6,M7,M8,M9}
54     // S0: load R, G, B into r1, r2, r3.
55     `S0: begin
56         control = 22'b1_1_1_0_0_0_000_00_00_00_0_00_0_0_0_0;
57         done = 1'b0;
58     end
59
60     // t1 = R * 0.299 -> r4
61     `S1: begin
62         control = 22'b0_0_0_1_0_0_000_10_00_00_0_00_0_0_0_0;
63         done = 1'b0;
64     end
65
66     // t2 = G * 0.587 -> r5
67     `S2: begin
68         control = 22'b0_0_0_0_1_0_001_01_00_00_0_00_0_0_0_0;
69         done = 1'b0;
70     end
71
72     // TODO
73
74     // t6 = R * (-0.169) -> r5, t4 = t1 + t2 -> r4
75     `S3: begin
```

testbench.v

讀取bmp file的檔頭資訊，bmp file格式可以在網路上找到。

```
40     bmp_width = {bmp_data[21],bmp_data[20],bmp_data[19],bmp_data[18]};
41     bmp_hight = {bmp_data[25],bmp_data[24],bmp_data[23],bmp_data[22]};
42     data_start_index = {bmp_data[13],bmp_data[12],bmp_data[11],bmp_data[10]};
43     bmp_size = {bmp_data[5],bmp_data[4],bmp_data[3],bmp_data[2]};
44
45     for(j = 0; j < 54; j = j + 1) begin
46         #(clk_period*1)
47         tempY = bmp_data[j];
48         $fwrite(imageOUTY,"%c",bmp_data[j]);
49         $fwrite(imageOUTU,"%c",bmp_data[j]);
50         $fwrite(imageOUTV,"%c",bmp_data[j]);
51     end
52
53     #(clk_period*1)
54     start= 1'b1;
55     #(clk_period*1)
```




實驗結果



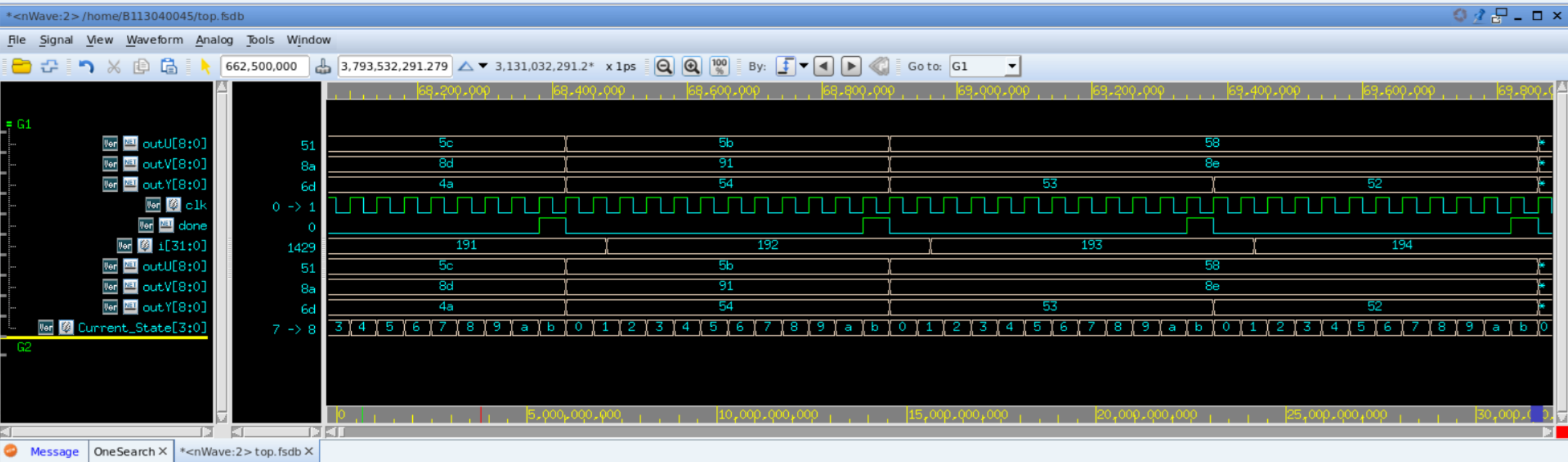
Y



U



V



signal

實驗心得

這次實驗是上次實驗的延伸，上次是撰寫排程演算法，而這次則是使用Verilog來撰寫電路。我覺得這次實驗最麻煩的地方在於架設環境與編譯，因為一群人同時使用伺服器，導致伺服器不堪負荷，光下載幾MB就要等待許久的時間。而且也不是使用之前必修課的軟體 - vivado來編譯，重新認識一個軟體花了不少的時間。

而至於程式碼的部分，因為上次已經研究過相同的演算法了，所以這部份不用重新了解，只要按照PPT的內容把電路邏輯撰寫進程式碼即可。